

04장 스레드와 프로세스 간 통신

학습목표

- 프로세스와 스레드의 차이점을 이해하고, 스레드를 사용하는 이유를 학습한다.
- 멀티스레딩 모델별 특징을 살펴본다.
- 프로세스 간 통신 기법을 학습하고, 각 기법을 비교하여 특징을 이해한다

목차

1. 스레드의 개념과 특징
2. 스레딩
3. 프로세스 간 통신
4. 프로세스 간 통신 코드

01

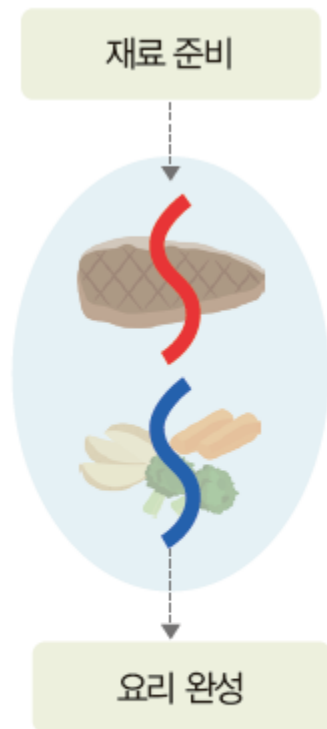
스레드의 개념과 특징

01 스레드의 개념과 특징

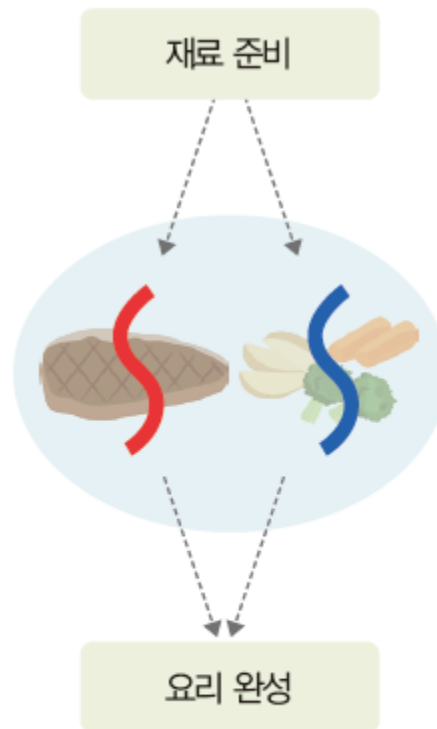
1. 스레드의 개념

◆ 스레드의 필요성

- 스레드: 프로세스 내부에서 실처럼 이어지는 실행 단위.



(a) 단일 스레드



(b) 멀티스레드

여러 개의 실행 단위가
작업을 병렬적으로 진행한다.



그림 4-1 스레드의 개념

01 스레드의 개념과 특징

1. 스레드의 개념

◆ 스레드와 프로세스의 관계

- 스레드: 프로세스의 코드에 정의된 절차에 따라 CPU 작업을 요청하는 실행 단위.
- 프로세스: 운영체제 입장에서의 작업 단위로, 하나 이상의 실행 단위(스레드)를 담는 틀인 단위.

표 4-1 프로세스와 스레드의 비교

구분	프로세스	스레드
정의	실행 중인 프로그램의 작업 단위	프로세스 내의 실행 단위
메모리	독립적인 메모리 공간을 가짐	같은 프로세스의 스레드들이 메모리를 공유함
실행 흐름	실행 흐름이 하나임	여러 개의 실행 흐름을 만들 수 있음

01 스레드의 개념과 특징

1. 스레드의 개념

◆ 멀티태스킹과 멀티스레딩

- **멀티태스킹**: 서로 독립적인 프로세스가 동시에 각자의 작업을 수행하는 방식.
 - 프로세스 간 통신 사용
- **멀티스레딩**: 하나의 프로세스 내부에서 여러 개의 스레드가 동시에 작업하는 방식.



(a) 멀티태스킹



(b) 멀티스레딩

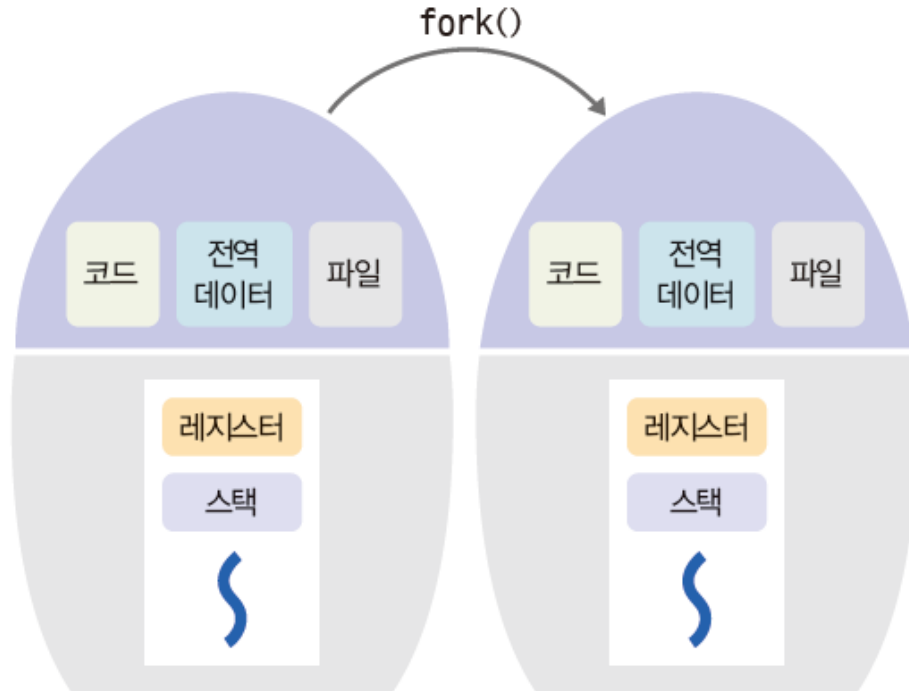
그림 4-2 멀티태스킹과 멀티스레딩

01 스레드의 개념과 특징

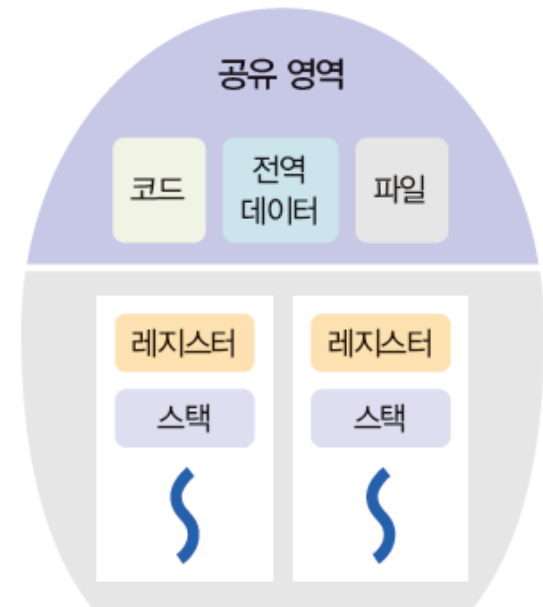
2. 멀티스레드의 장단점

◆ 멀티스레드의 구조

- 멀티태스킹과 멀티스레드의 구조적 차이



(a) 멀티태스킹



(b) 멀티스레딩

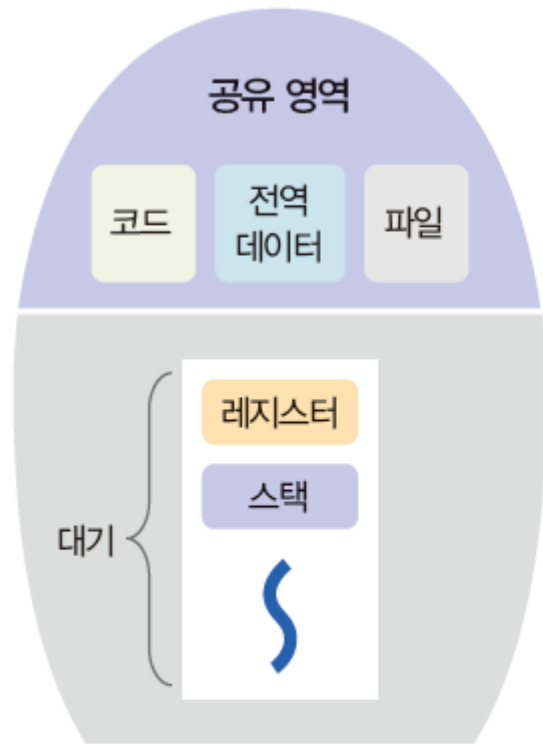
그림 4-3 프로세스의 구조

01 스레드의 개념과 특징

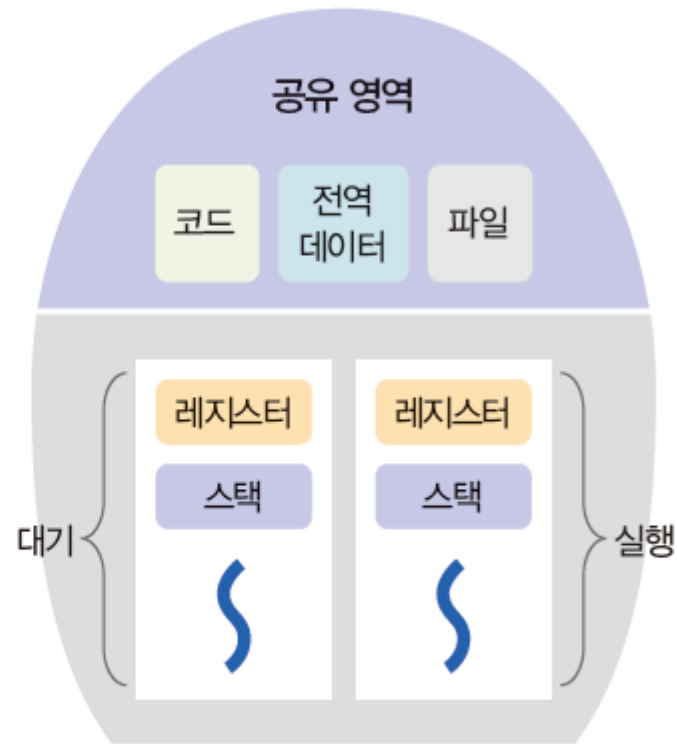
2. 멀티스레드의 장단점

◆ 멀티스레드의 구조

- 비디오 플레이어 프로세스



(a) 단일 스레드



(b) 멀티스레드

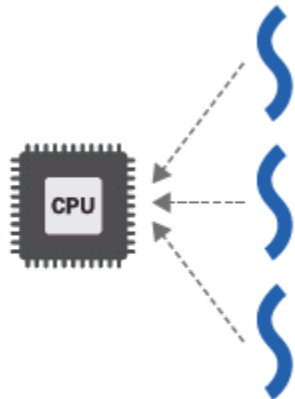
그림 4-4 비디오 플레이어 프로세스의 상태

01 스레드의 개념과 특징

2. 멀티스레드의 장단점

◆ 멀티스레드의 구조

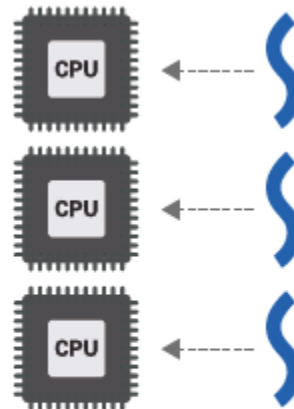
- 병렬처리 용어



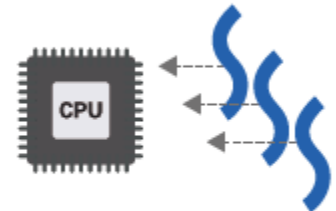
(a) 멀티태스킹(시분할)



(b) 멀티스레드



(c) 멀티프로세싱



(d) CPU 멀티스레딩

그림 4-5 스레드 관련 개념

01 스레드의 개념과 특징

2. 멀티스레드의 장단점

◆ 멀티스레드의 장점

■ 자원공유

- 여러 개의 스레드가 코드, 데이터, 파일등의 자원을 공유하여 낭비를 최소화.

■ 문맥 교환

- 스레드가 프로세스 내부에서 실행되어 변경할 정보가 적고, 프로세스 간 통신이 없어도 데이터를 주고받을 수 있어 통신 비용과 속도가 감소.

■ 응답 속도

- 입출력 담당 스레드만 대기 상태, 나머지는 계속 실행 가능하여 응답속도 향상.

■ 적응성

- 여러 개의 코어에 스레드를 분배하여 멀티코어 환경 최적화.

01 스레드의 개념과 특징

2. 멀티스레드의 장단점

◆ 멀티스레드의 단점

- 모든 스레드가 프로세스와 함께 종료



그림 4-6 크롬 웹 브라우저의 멀티태스킹

02

스레딩

02 스레딩

1. 멀티스레딩 모델

◆ 1 to N 모델: 사용자 스레딩

- 낮은 문맥 교환 비용
 - 커널의 개입 없이 스레드를 전환, 부가적 작업X
- 입출력 대기 비효율
 - 커널 스레드가 입출력 작업을 위해 대기 상태에 들어가면 모든 사용자 스레드가 멈춤
- 멀티코어 활용 불가능
 - 사용자 스레드 여러 개가 하나의 프로세스로 인식되어 스레드의 분산이 불가.
- 보안 취약
 - 공유 변수 보호 등의 기능을 스레드 라이브러리에 구현해야 하므로 안정성이 낮음.

02 스레딩

1. 멀티스레딩 모델

◆ 1 to N 모델: 사용자 스레딩

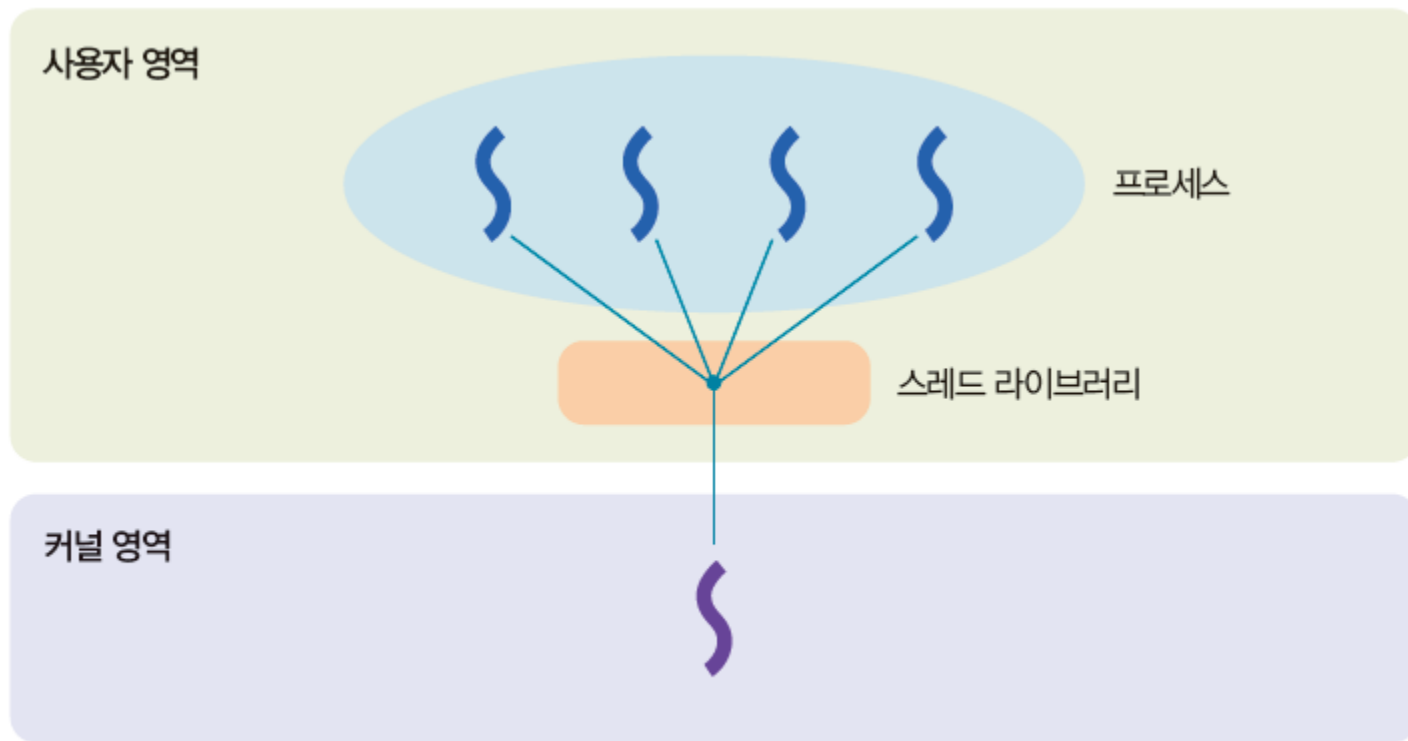


그림 4-7 1 to N 모델

02 스레딩

1. 멀티스레딩 모델

◆ 1 to 1 모델: 커널 스레딩

- 높은 문맥 교환 비용

- 커널이 직접 스레드 관리, 문맥 교환 시 오버헤드 발생.

- 입출력 대기 효율

- 커널 스레드가 직접 스레드를 스케줄링, 대기 상태에서도 다른 스레드가 작업 수행.

- 멀티코어 활용 가능

- 운영체제가 스레드를 여러 개의 CPU에 분산하여 실행 가능.

- 안정성

- 커널에서 공유 변수 보호 기능을 제공, 사용자 스레드보다 데이터 충돌과 보안 문제 적음.

02 스레딩

1. 멀티스레딩 모델

◆ 1 to 1 모델: 커널 스레딩

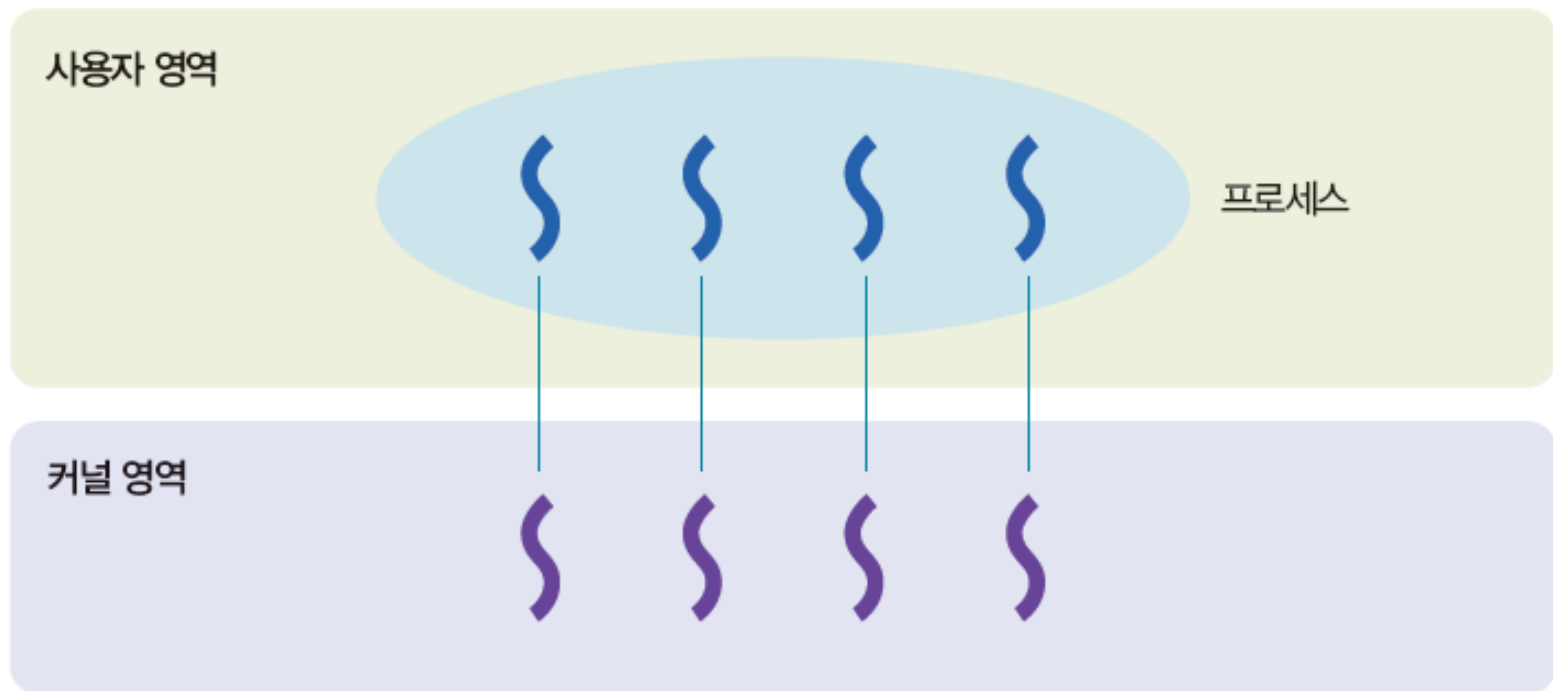


그림 4-8 1 to 1 모델

02 스레딩

1. 멀티스레딩 모델

◆ M to N 모델: 멀티레벨 스레딩

■ 높은 문맥 교환 비용

- 커널 스레드를 공동으로 사용, 문맥 교환 시 오버헤드가 있어 사용자 스레드만큼 빠르지 않음.

■ 입출력 대기 유연성

- 하나의 커널 스레드가 대기 상태일 때 다른 커널 스레드가 대신 작업하여 유연하게 작업 처리.

■ 멀티코어 활용 가능

- 운영체제가 스레드를 여러 개의 CPU에 분산하여 실행 가능.

■ 스케줄링 및 동기화의 복잡성

- 사용자 스레드와 커널 스레드의 혼합 모델이므로 스케줄링, 동기화가 복잡.

02 스레딩

1. 멀티스레딩 모델

◆ M to N 모델: 멀티레벨 스레딩

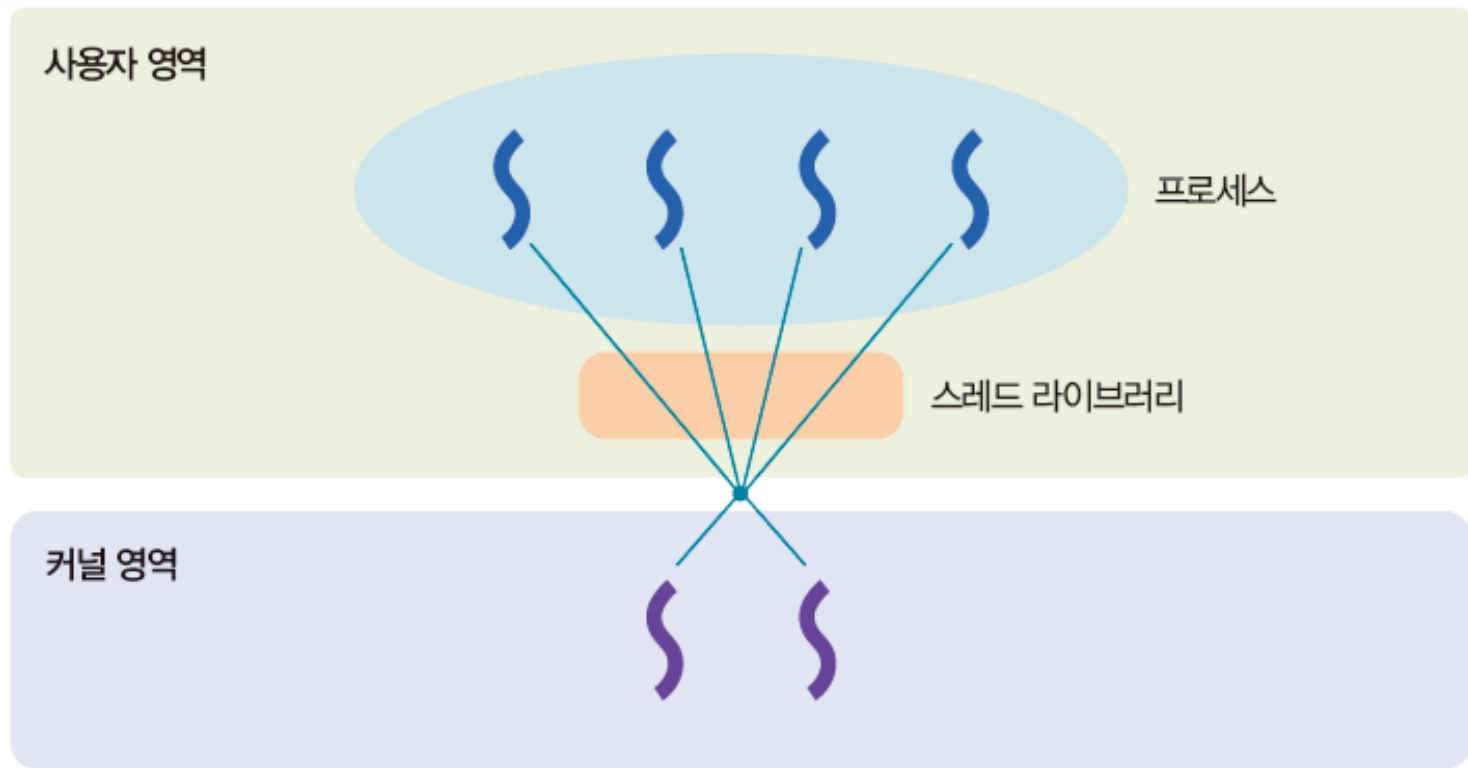


그림 4-9 M to N 모델

02 스레딩

2. [심화] 스레드 코드

◆ 유닉스에서 스레드 만들기

- 스레드를 만드는 함수 pthread_create()
- 만들어진 스레드를 기다리는 함수 Pthread_join()
- Fork() = create() , wait() = join()

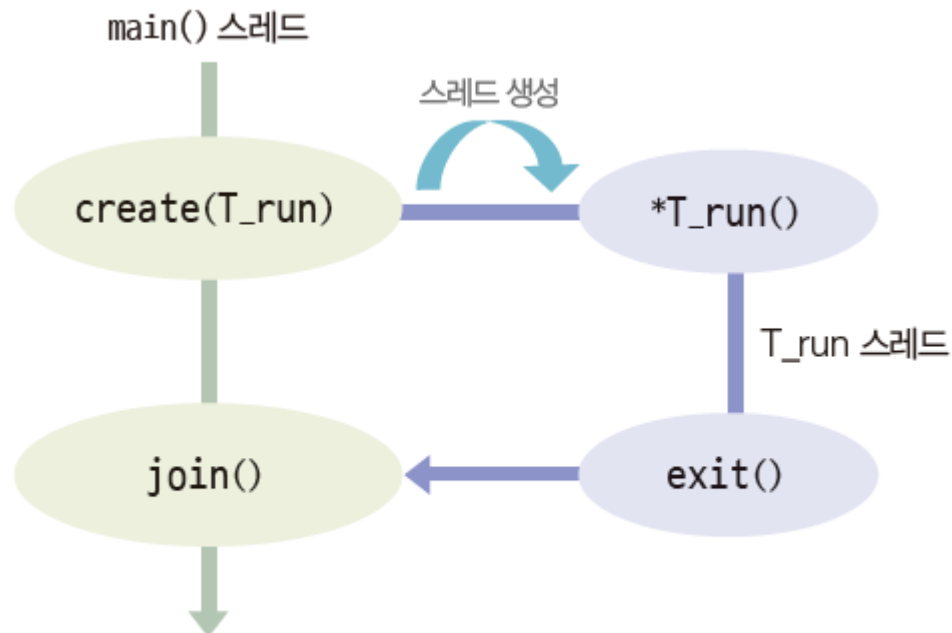


그림 4-10 스레드 관련 시스템 호출

02 스레딩

코드 4-1 pthread 코드

```
01 #include <stdio.h>
02 #include <pthread.h>
03
04 int sum = 0;
05
06 void *T_run() {                // 스레드 실행
07     for (int i = 1; i < 5; i++)
08         printf("Td: %d add -> sum = %d\n", i, sum = sum + i);
09     pthread_exit(NULL);        // 스레드 종료
10 }
11
12 int main() {
13     pthread_t tid;              // 스레드 구조체
14
15     pthread_create(&tid, NULL, T_run, NULL); // 스레드 생성 -> T_run()
16     printf("Main: sum = %d\n", sum);
17     pthread_join(tid, NULL);     // 스레드 기다림 - 자원회수
18     printf("Main: sum = %d\n", sum);
19
20     return 0;
21 }
```

02 스레딩

2. 스레드 코드

◆ 유닉스에서 스레드 만들기

- Pthread 라이브러리를 사용한 코드

```
Main: sum = 0
Td: 1 add -> sum = 1
Td: 2 add -> sum = 3
Td: 3 add -> sum = 6
Td: 4 add -> sum = 10
Main: sum = 10
```

02 스레딩

2. 스레드 코드

◆ 자바에서 스레드 만들기

- 자바는 별도의 라이브러리 없이 자체적인 스레드 지원

02 스레딩

코드 4-2 자바 스레드 코드

```
01  class TH_test extends Thread {           // TH_test 스레드 클래스 정의
02      public void run() {                   // run() 메서드는
03          for (int i = 0; i < 5; i++)       // "Thread" 문자열을 5번 출력
04              System.out.println("Thread");
05      }
06  }
07
08  public class Main {
09      public static void main(String[] args) {
10          TH_test TH_print = new TH_test(); // 스레드 객체 생성
11
12          TH_print.start();                 // 멀티스레드 실행
13
14          for (int i = 0; i < 5; i++)
15              System.out.println("* main");
16      }
17  }
```


02 스레딩

2. 스레드 코드

◆자바에서 스레드 만들기

- 결과

```
* main
Thread
Thread
Thread
Thread
Thread
* main
* main
* main
* main
```

02 스레딩

2. 스레드 코드

◆자바에서 스레드 동기화하기

- 앞의 자바 스레드 코드는 동기화 과정이 없어 main()과 스레드가 동시 실행.
- join()을 사용하여 동기화 시, 스레드가 끝난 후 main()이 실행되도록 제어 가능.
 - try-catch 방식이 일반적.

코드 4-3 join()을 사용하는 자바 스레드 동기화 코드

```
01 class TH_test extends Thread {  
02     public void run() {  
03         for (int i = 0; i < 5; i++)  
04             System.out.println("Thread");  
05     }  
06 }  
07
```

02 스레딩

2. 스레드 코드

◆자바에서 스레드 동기화

```
08 public class Main {
09     public static void main(String[] args) {
10         TH_test TH_print = new TH_test();
11
12         TH_print.start();
13         try {                                     // 동기화
14             TH_print.join();
15         } catch (InterruptedException e) {}
16
17         for (int i = 0; i < 5; i++)
18             System.out.println("* main");
19     }
20 }
```

02 스레딩

2. 스레드 코드

◆자바에서 스레드 동기화

```
Thread  
Thread  
Thread  
Thread  
Thread  
* main  
* main  
* main  
* main  
* main
```

03

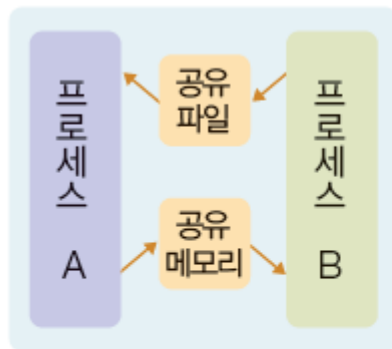
프로세스 간 통신

03 프로세스 간 통신

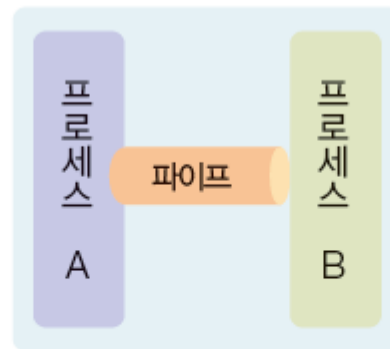
1. 프로세스 간 통신의 개념

- 프로세스 간 통신(IPC): 프로세스가 서로 연결을 주고받는 것
- 운영체제는 프로세스끼리 데이터를 주고받도록 다양한 프로세스 통신기법 제공

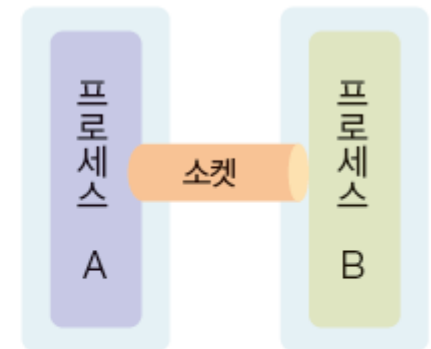
◆ 프로세스 간 통신 기법의 종류



(a) 공유 메모리 및 공유 파일



(b) 파이프 통신



(c) 소켓 통신

그림 4-11 프로세스 간 통신 기법

03 프로세스 간 통신

1. 프로세스 간 통신의 개념

◆ 프로세스 간 통신 기법의 종류

표 4-2 프로세스 간 통신 기법의 종류

구분	설명	데이터 전달 방식
공유 메모리	일정한 메모리 영역을 공유하여 데이터 전달	프로세스가 직접 처리
공유 파일	동일한 파일을 사용하여 데이터 교환	프로세스가 직접 처리
파이프	동일한 시스템 내에서 데이터 전달	운영체제가 처리
소켓	네트워크 또는 동일한 시스템 내에서 데이터 전달	운영체제가 처리

◆ 데이터 전달 방향에 따른 통신 분류

- 단방향(simplex) 통신
- 양방향(duplex) 통신
- 반양방향(half duplex) 통신

03 프로세스 간 통신

1. 프로세스 간 통신의 개념

◆ 동기화 통신과 비동기화 통신

■ 비동기화 통신

- 데이터 도착 시점을 알 수 없어, 프로세스가 반복문을 통해 도착 여부를 직접 확인 [바쁜 대기]
- 논 블로킹

■ 동기화 통신

- 동기화 통신은 운영체제가 데이터 도착까지 자동으로 대기시킴
- 블로킹 방식의 통신
- 바쁜 대기를 사용할 필요가 없어, 데이터가 도착한 이후에 다음 코드 실행

03 프로세스 간 통신

2. 프로세스 간 통신의 기법

◆ 프로세스의 파일 접근

- 파일의 입출력 방식

권한이 있으면 키를 얻음



(a) `fd = open()`

읽기 및 쓰기 작업



(b) `read(fd)`, `write(fd)`

키 반납



(c) `close(fd)`

그림 4-12 파일 입출력 방법

03 프로세스 간 통신

코드 4-4 파일 읽기 코드

```
01  #include <stdio.h>
02  #include <unistd.h>
03  #include <fcntl.h>
04
05  void main() {
06      int fd, n;
07      char buf[6];
08
09      fd = open("test.txt", O_RDONLY);    // 파일 open
10
11      if (fd == -1) {                    // 파일 열기 실패
12          perror("Error opening file");
13          return 1;
14      }
15
```

03 프로세스 간 통신

```
16     n = read(fd, buf, 5);           // read(fd): n은 실제 읽은 문자 수
17     if (n > 0) {
18         buf[n] = '\0';               // 맨 마지막에 NULL 문자 삽입
19         printf("%s\n", buf);
20     } else {
21         perror("Error reading file");
22     }
23     close(fd);                       // 파일 close
24 }
```

03 프로세스 간 통신

2. 프로세스 간 통신의 기법

◆파이프 통신

- 운영체제가 제공하는 동기화 방식의 프로세스 간 통신 기법.

코드 4-5 파이프 통신 코드(보내는 쪽)

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int fd[2];    // 2개의 파일 기술자: 읽기용 fd[0], 쓰기용 fd[1]

    pipe(fd);     // 파이프 생성

    close(fd[0]);    // 읽기용 기술자 닫기
    write(fd[1], "TEST", 5); // fd[1]을 통해 데이터 쓰기
    close(fd[1]);    // 쓰기용 기술자 닫기

    return 0;
}
```

03 프로세스 간 통신

2. 프로세스 간 통신의 기법

◆파이프 통신

- 파일 입출력과 유사하게 동작하지만 `open()` 대신 `pipe(fd)`를 사용
 - `pipe()`-`read()`와 `write()`-`close()` 형태.
- 단방향 통신만을 지원.
- 1개의 파이프에 2개의 파이프 기술자 사용.

03 프로세스 간 통신

2. 프로세스 간 통신의 기법

◆파이프 통신

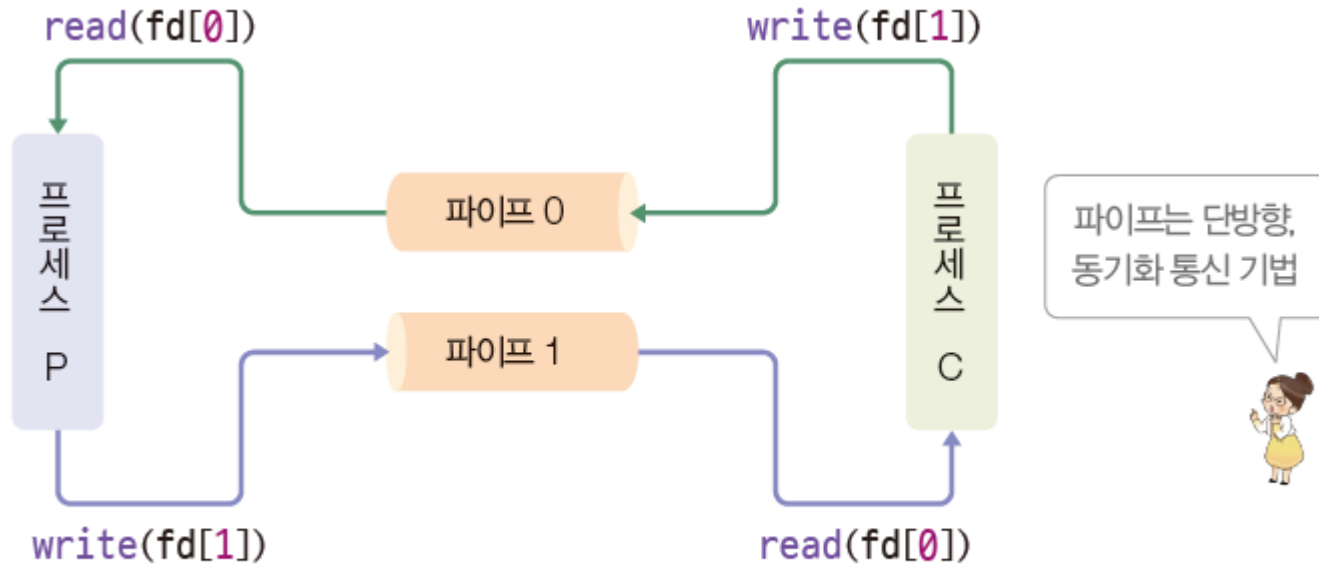


그림 4-13 파이프를 이용한 통신

- 파이프 통신에서 부모 프로세스는 바쁜 대기(busy wait)를 하지 않아도 됨.

03 프로세스 간 통신

2. 프로세스 간 통신의 기법

◆ 네트워크 통신 이해하기

- IP: 데이터를 목적지까지 전송하기 위해 개발된 통신 규약.
- TCP: 네트워크를 통해 받은 데이터를 정리, 인터페이스의 역할을 하는 통신 규약.



그림 4-14 IP 주소와 포트 번호

03 프로세스 간 통신

2. 프로세스 간 통신의 기법

◆ 네트워크 통신 이해하기

- 포트 번호: TCP가 네트워크를 사용하는 프로세스를 구분하기 위해 사용하는 주소.
- 소켓 프로그래밍: 하나의 포트에 여러 클라이언트가 동시에 접속할 수 있도록 소켓이 중간에서 연결을 관리하는 통신 방식.

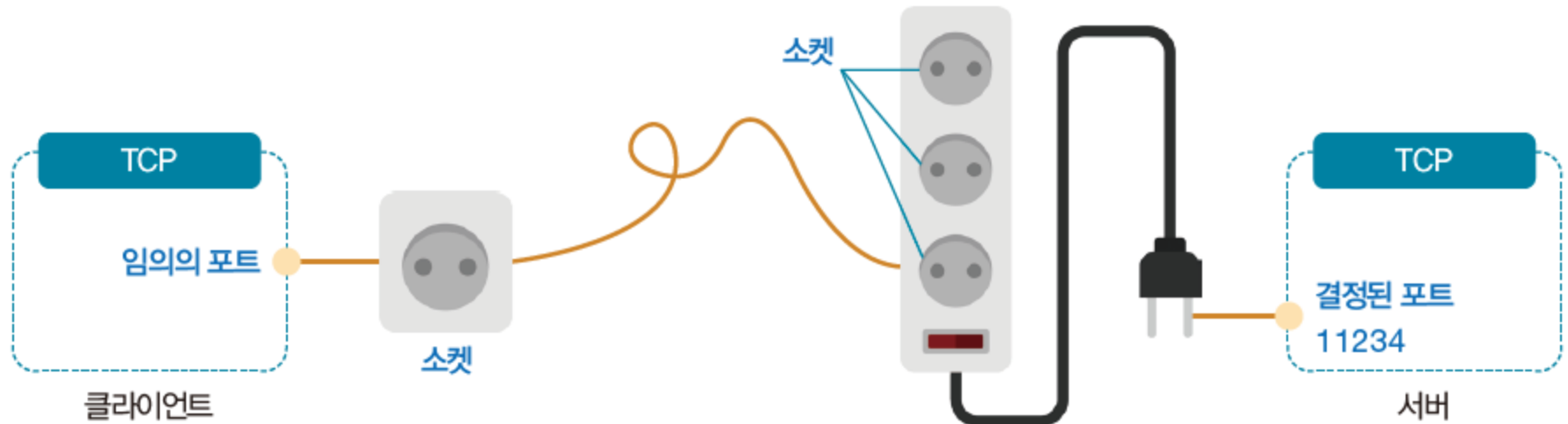


그림 4-15 클라이언트-서버 통신에서의 소켓

03 프로세스 간 통신

2. 프로세스 간 통신의 기법

◆소켓 통신

- 양방향 데이터 송수신이 가능하며, 동기화를 지원.
- 바쁜 대기가 필요없음.
- `cs=socket()`으로 소켓이 만들어지며, 여기서 `cs`는 소켓 기술자(socket descriptor).

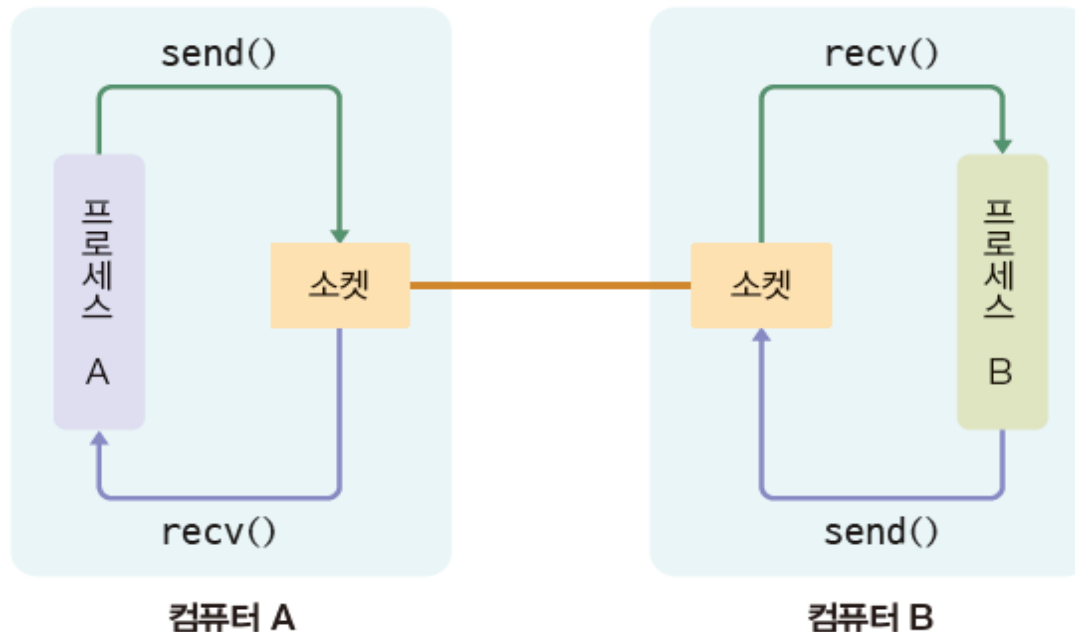


그림 4-16 소켓을 이용한 통신

04 [심화]

프로세스 간 통신 코드

04 프로세스 간 통신 코드

1. 파일 입출력

◆ 파일 포인터

- 순차 파일 구조는 데이터가 하나의 긴 줄로 이어진 형태.

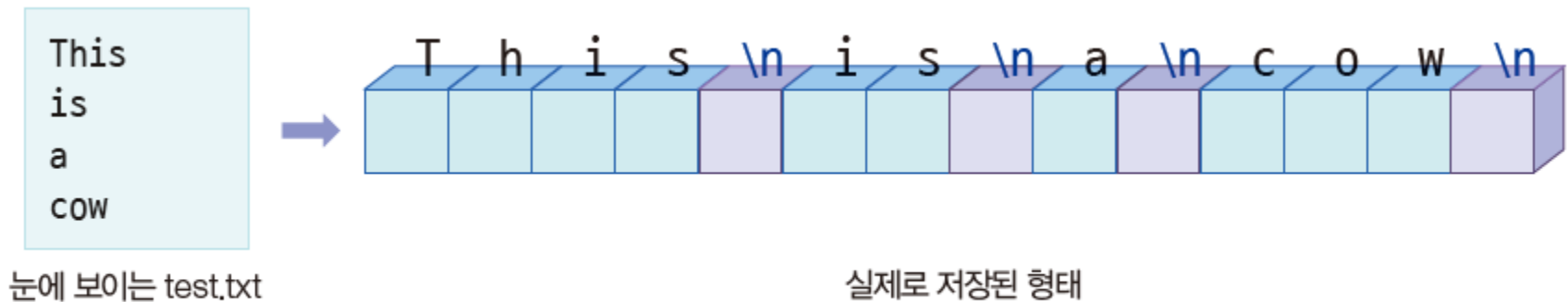


그림 4-17 순차 파일 구조

04 프로세스 간 통신 코드

1. 파일 입출력

◆파일 포인터

- 파일에서는 파일 포인터(파일 오프셋, 위치 지시자)가 작업 위치를 가리킴.
- 파일 시작부터 얼마나 떨어져 있는지를 바이트 단위로 나타냄.

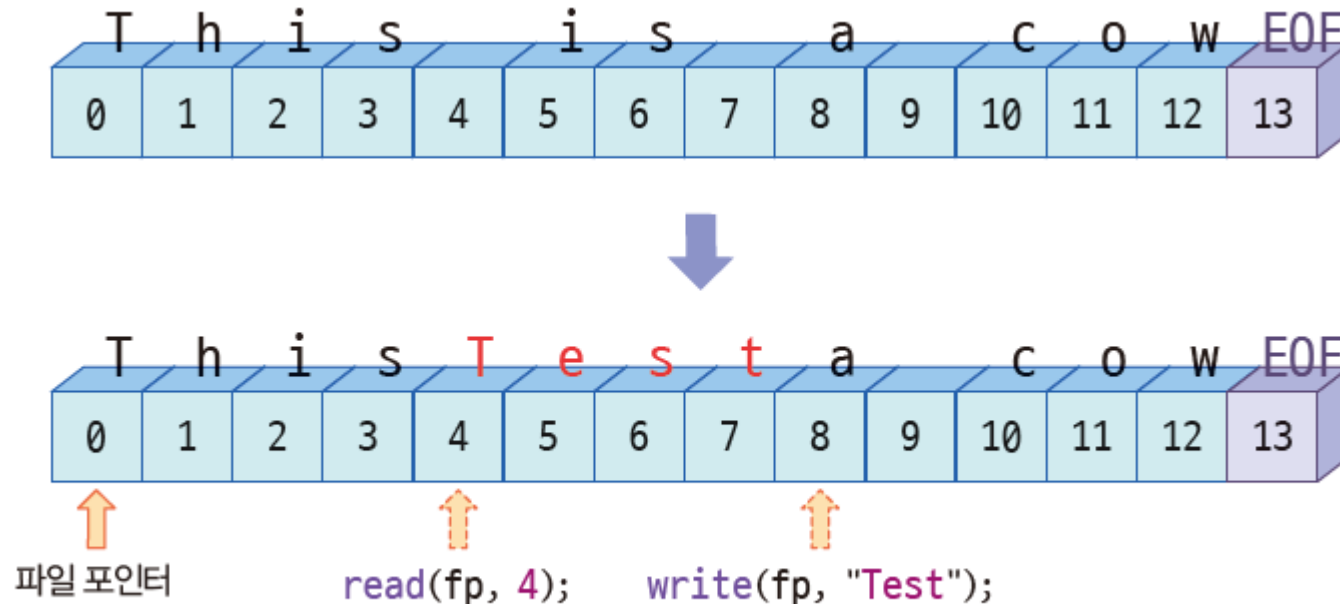


그림 4-18 파일 포인터의 이동

04 프로세스 간 통신 코드

1. 파일 입출력

◆fork()와 파일 기술자

코드 4-6 fork()와 파일 접근 코드

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  #include <unistd.h>
04  #include <fcntl.h>
05  #include <sys/wait.h>
06
07  void main() {
08      int fd, pid, n;
09      char buf[6];
10
11      fd = open("test.txt", O_RDWR);    // ORDWR - 읽기 쓰기
12      pid = fork();                     // 자식 프로세스에게 파일 기술자 상속
13  }
```

04 프로세스 간 통신 코드

1. 파일 입출력

◆fork()와 파일 기술자

```
14     if (pid == 0) {                // 자식 프로세스
15         write(fd, "TEST", 5);
16         close(fd);                  // 자식 파일 기술자 닫음
17         exit(0);
18     } else {                        // 부모 프로세스
19         wait(0);                    // 자식을 기다림
20         lseek(fd, 0, SEEK_SET);     // 파일 포인터를 처음으로 이동
21         n = read(fd, buf, 5);
22         buf[n] = '\0';
23         printf("%s \n", buf);
24         close(fd);                  // 부모 파일 기술자 닫음
25     }
26 }
```

04 프로세스 간 통신 코드

2. 파이프 코드를 통한 부모-자식 프로세스 간 통신

◆파이프의 양방향 통신

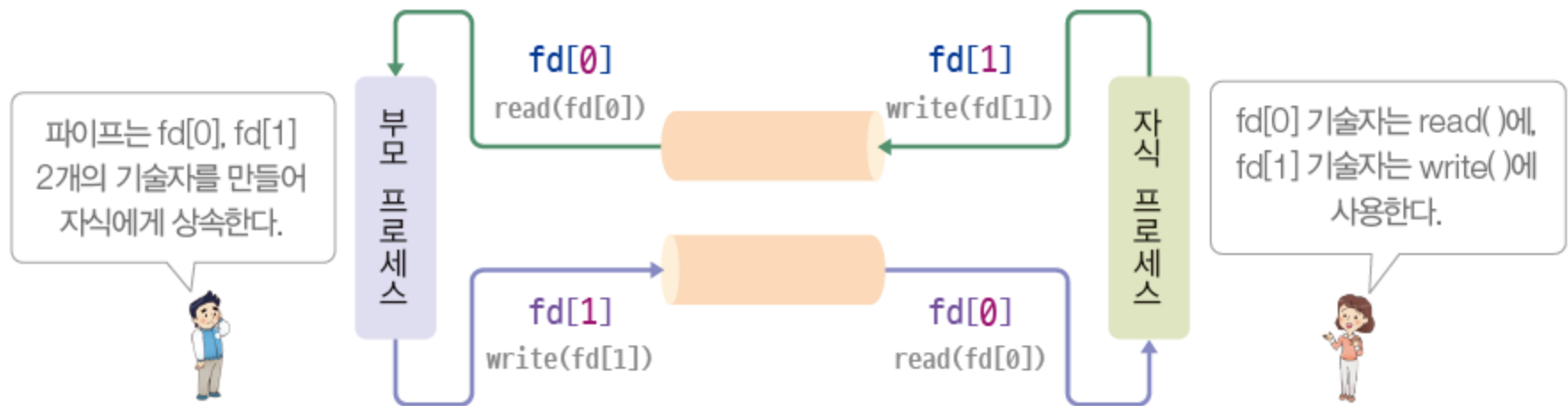


그림 4-19 파이프 통신의 구조

04 프로세스 간 통신 코드

2. 파이프 코드를 통한 부모-자식 프로세스 간 통신

◆파이프의 양방향 통신

코드 4-7 파이프 통신 코드

```
01  #include <stdio.h>
02  #include <unistd.h>
03
04  void main() {
05      pid_t pid;
06      int fd[2], n;           // 2개의 파일 디스크립터 fd[0] 읽기, fd[1] 쓰기
07      char buf[6];
08
09      pipe(fd);               // 파이프 생성
10      pid = fork();           // 자식 프로세스에게 파이프 기술자 상속
11
```


04 프로세스 간 통신 코드

```
12     if (pid == 0) {                // 자식 프로세스
13         close(fd[0]);              // close fd[0]
14         write(fd[1], "TEST", 5);   // write to fd[1]
15         close(fd[1]);
16     } else {                        // 부모 프로세스
17         close(fd[1]);              // close fd[1]
18         n = read(fd[0], buf, 5);    // read from fd[0]
19         buf[n] = '\0';
20         printf("Parent receive = %s\n", buf);
21         close(fd[0]);
22     }
23 }
```

Parent receive = TEST

04 프로세스 간 통신 코드

3. 소켓 코드를 통한 네트워크 기반 통신

◆ 소켓 코드의 구조

- `cs = socket()`을 호출하면 소켓 기술자를 반환하고, 이후 모든 작업은 소켓 기술자(cs)를 통해 이루어짐.

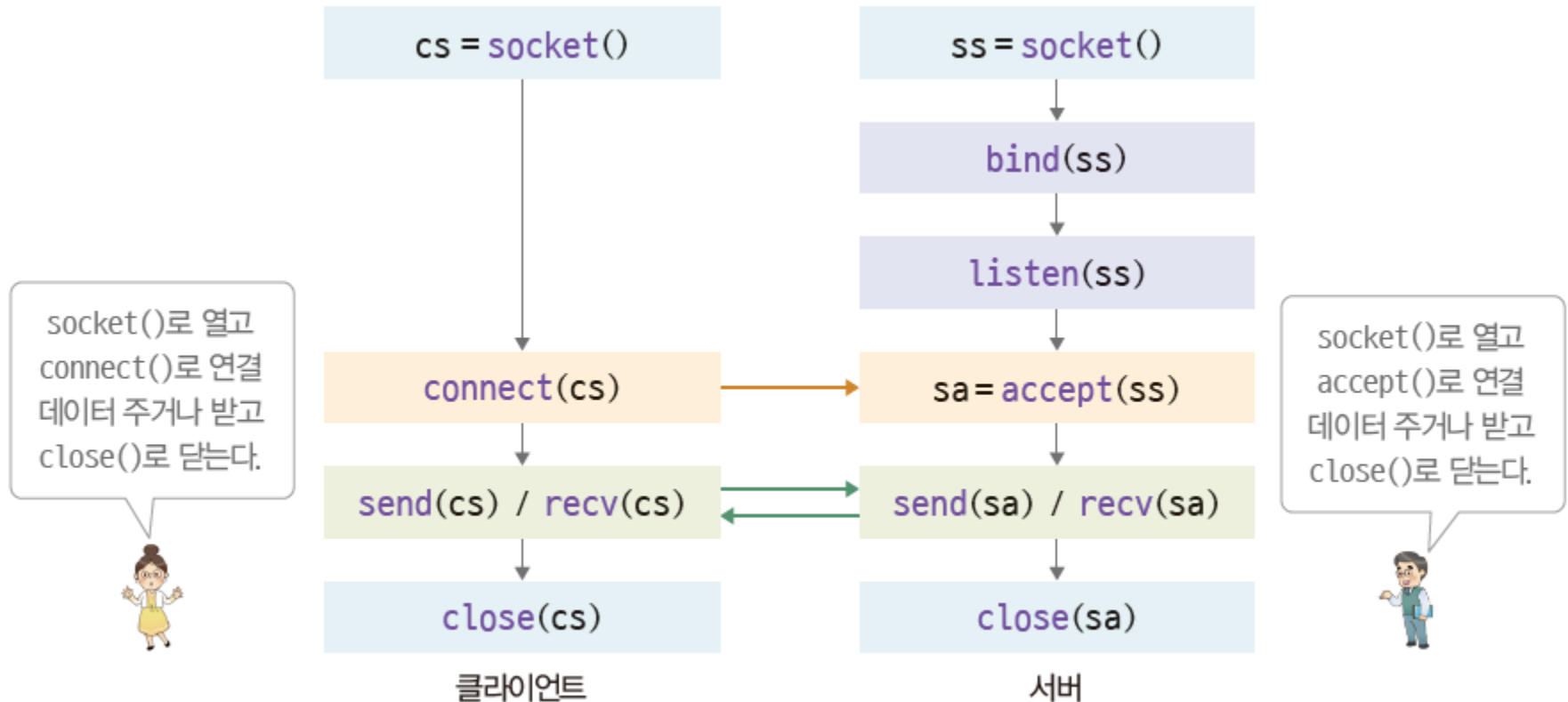


그림 4-20 클라이언트 소켓과 서버 소켓의 통신 절차

04 프로세스 간 통신 코드

3. 소켓 코드를 통한 네트워크 기반 통신

◆클라이언트 코드 분석

코드 4-8 클라이언트 소켓 코드

```
01  #include <stdio.h>                // printf()
02  #include <string.h>               // memset()
03  #include <unistd.h>               // close()
04  #include <sys/socket.h>           // socket library
05  #include <arpa/inet.h>           // internet library
06
07  void main() {
08      int cs;
09      char buf[5];
10      struct sockaddr_in csa;        // 서버 주소 구조체
11
```

04 프로세스 간 통신 코드

3. 소켓 코드를 통한 네트워크 기반 통신

◆클라이언트 코드 분석

```
12  memset(&csa, 0, sizeof(csa));           // 주소 구조체 초기화
13  csa.sin_family = AF_INET;               // 인터넷 소켓
14  csa.sin_addr.s_addr = inet_addr(127.0.0.1); // 서버 IP(루프 백 주소)
15  csa.sin_port = htons(11234);            // 서버 포트 번호 11234
16
17  cs = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP); // 소켓 생성
18  connect(cs, (struct sockaddr *)&csa, sizeof(csa)); // 서버 연결 요청
19
20  recv(cs, buf, 5, 0);
21  printf("Receive [%s]\n", buf);
22
23  close(cs);                               // 소켓 종료
24 }
```

04 프로세스 간 통신 코드

3. 소켓 코드를 통한 네트워크 기반 통신

◆ 서버 코드 분석

코드 4-9 서버 소켓 코드

```
01  #include <string.h>                // memset()
02  #include <unistd.h>                // close()
03  #include <sys/socket.h>            // socket library
04  #include <arpa/inet.h>            // internet library
05
06  void main() {
07      int ss, sa;
08      struct sockaddr_in ssa;        // 서버 주소 구조체
09
```

04 프로세스 간 통신 코드

3. 소켓 코드를 통한 네트워크 기반 통신

◆ 서버 코드 분석

```
10  memset(&ssa, 0, sizeof(ssa));
11  ssa.sin_family = AF_INET;           // 인터넷 소켓
12  ssa.sin_addr.s_addr = htonl(INADDR_ANY); // 모든 IP 주소 허용
13  ssa.sin_port = htons(11234);        // 포트 번호 11234
14
15  ss = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP); // 소켓 생성
16  bind(ss, (struct sockaddr *) &ssa, sizeof(ssa)); // 소켓 바인딩
17  listen(ss, 10);                       // 최대 10개 클라이언트
18
19  while(1) {                             // 무한 루프 - 계속 실행
20      sa = accept(ss, 0, 0);             // 클라이언트 요청 수락
21      send(sa, "test", 5, 0);
22      close(sa);                         // 클라이언트 소켓 종료
23  }
24 }
```

04 프로세스 간 통신 코드

3. 소켓 코드를 통한 네트워크 기반 통신

◆ 서버 코드 분석

- 클라이언트가 몇 번을 접속하더라도 서버는 계속 살아남아서 클라이언트에게 문자열 "test"를 보냄.
- `close(ss)`를 사용한다면 서버가 서비스를 종료하겠다는 의미.

04 프로세스 간 통신 코드

3. 소켓 코드를 통한 네트워크 기반 통신

◆ 서버 코드 분석

<code>\$>gcc -o server server.c</code>	서버 코드 컴파일
<code>\$>gcc -o client client.c</code>	클라이언트 코드 컴파일
<code>\$>server &</code>	서버 코드를 백그라운드에서 실행
<code>\$>client</code>	클라이언트 실행
<code>\$>Receive [test]</code>	서버로부터 test 수신
<code>\$>client</code>	클라이언트 다시 실행
<code>\$>Receive [test]</code>	서버로부터 test 수신

그림 4-21 유닉스에서 서버와 클라이언트 소켓 코드의 실행 과정

- 루프백 주소를 사용하여 서버와 클라이언트 코드가 같은 컴퓨터에서 실행되기 때문에 서버는 백그라운드로 실행.
- 계속 살아있는 서버 프로세스를 중지하려면 ps 명령어를 사용하여 프로세스 구분자(PID)를 확인한 후 kill PID로 중지.

Thank You!